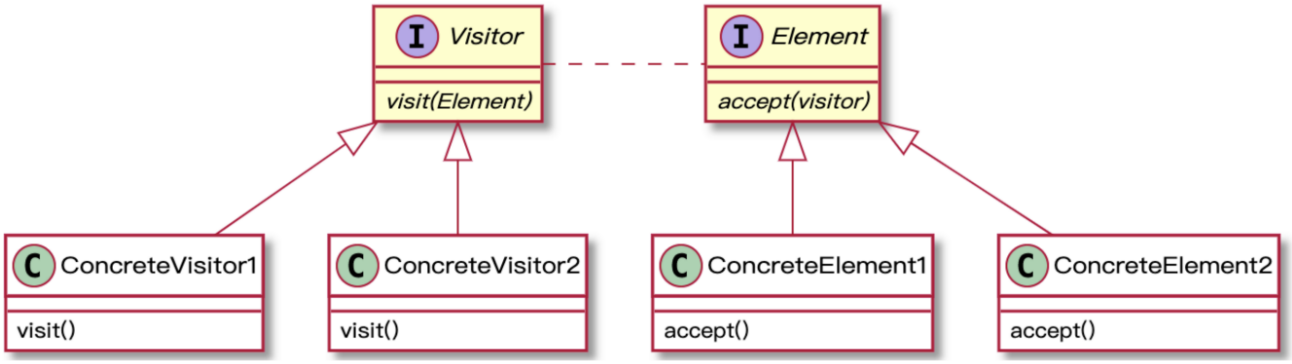
设计模式

Visitor



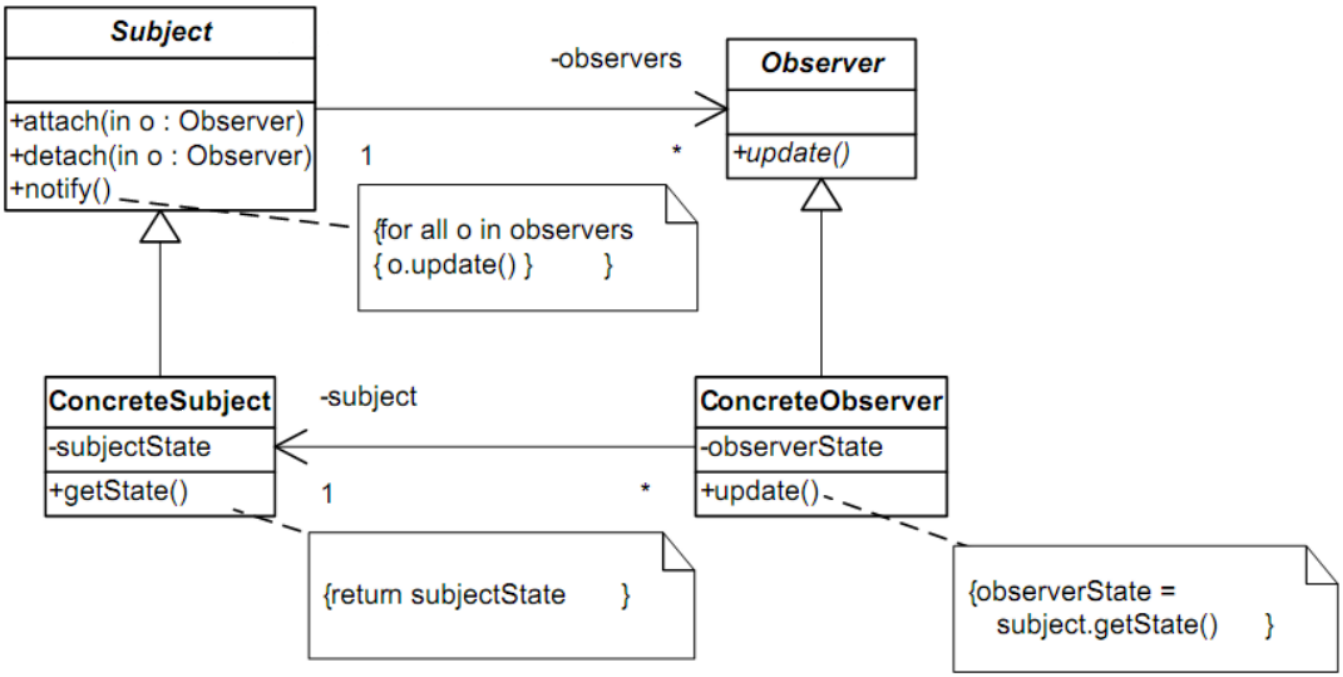
基本概念

- **使用场景**：适合对象类型固定，但需要不断给它们加新操作的场景
- **适用理由**：假如我原本有一个 `ConcreteElement`，如果我没有 Visitor，我要添加业务逻辑就只能在这个类里面加新的方法，这样很麻烦，因为多了之后一个类就会变得非常冗长。但是用了 Visitor 之后就只需要在 `ConcreteElement` 类里面加一个 `accept()` 方法，然后通过添加 `ConcreteVisitor` 来实现更多的功能，不需要修改 `ConcreteElement`。
- **工作流程**：首先，客户端创建具体的访问者 `ConcreteVisitor` 和一组元素对象 `ConcreteElement`；当需要对元素执行某种操作时，客户端将访问者传入元素的 `accept(visitor)` 方法；元素在 `accept` 中把自身 `this` 回调给访问者，调用对应的 `visit(ConcreteElementX)`；访问者根据元素的具体类型在自己已有的 `visit` 方法中找到适配的 `visit` 方法并执行相应的业务逻辑。

主要角色

- **Element 接口**：所有被访问对象的统一父接口
  - `accept(Visitor)`：抽象的 accept() 方法，规定 Element 可以被 Visitor 访问，自己没有任何其他操作
- **ConcreteElement1 / ConcreteElement2**：把对象的真实类型传给 Visitor
-
  - `accept()`：实现具体的 accept() 方法，把元素对象交给 Visitior 的操作
- **Visitor 接口**：定义我能访问哪些 Elements
  - `visit(Element)`：抽象的 visitor() 方法，规定 Visitor 可以被 Element 回调，自己没有任何其他操作
- **ConcreteVisitor1 / ConcreteVisitor2**：实现具体需要实现的逻辑，不同 Visitor 用来实现不同的业务逻辑
  - `visit()`：实现具体的 visit() 方法，这个方法用来实现具体的业务逻辑，也就是想要对 Element 进行的操作

Observer



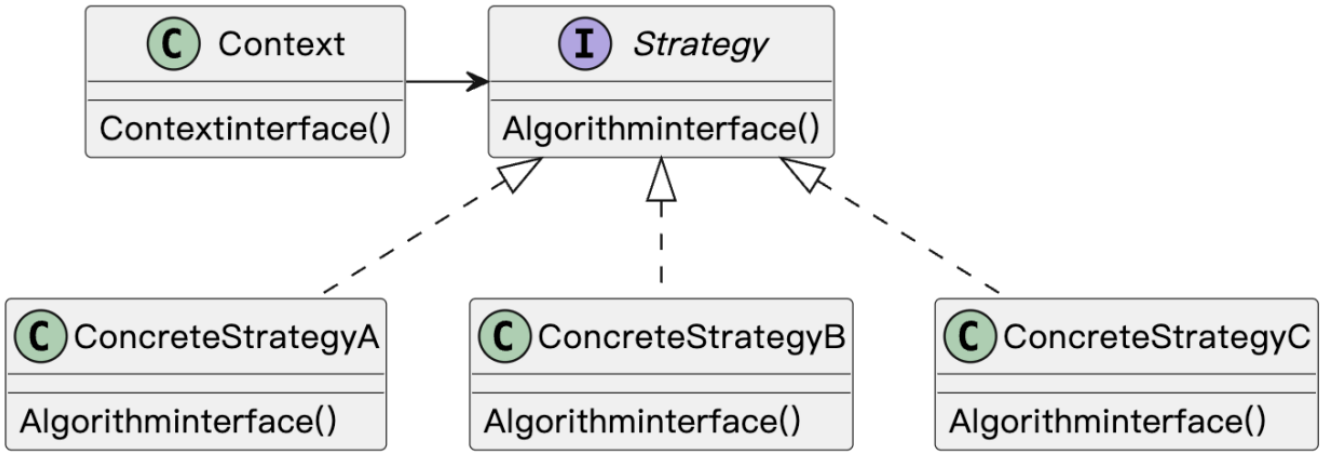
基本概念

- **使用场景**：日志、监控、订阅/发布系统
  - 一个对象的状态变化，需要通知多个对象
  - 不希望 Subject 直接依赖具体的接收方
  - 观察者的数量、类型可能动态变化
- **使用流程**：Observer 先向 Subject 注册自己，Subject 内部状态发生变化时调用 `notify()` 统一通知所有已注册的 Observer，每个 Observer 的 `update()` 随即被调用，Observer 在 `update()` 中通过 `getState()` 主动拉取 Subject 的 `subjectState` 并根据该状态独立完成 `observerState` 的业务更新

主要角色

- **Subject（被观察者接口 / 抽象类）**：管理观察者，在状态变化时通知观察者
  - `attach(o)`：注册观察者
  - `detach(o)`：移除观察者
  - `notify()`：当 Subject 遍历 Observers，调用 `o.update`。告诉 Observers Subject更新了，但没有响应的逻辑，响应的逻辑在 `o.update` 里面
- **Observer（观察者接口）**：定义当 Subject 变化时的响应接口
  - `update()`：用来执行更新后需要响应的具体逻辑，Subject 通知时被调用
- **ConcreteSubject（具体被观察者）**：保存真正的状态，提供状态访问接口
  - `subjectState`：被观察的核心数据
  - `getState()`：供 Observer 拉取状态
- **ConcreteObserver（具体观察者）**：保存自己关心的状态，在 `update()` 中从 Subject 拉数据
  - `observerState`：观察者自己的状态
  - `update()`：更新的具体业务逻辑，通过 `subject.getState()`，把 `observerState` 更新成 `subjectState`

Strategy



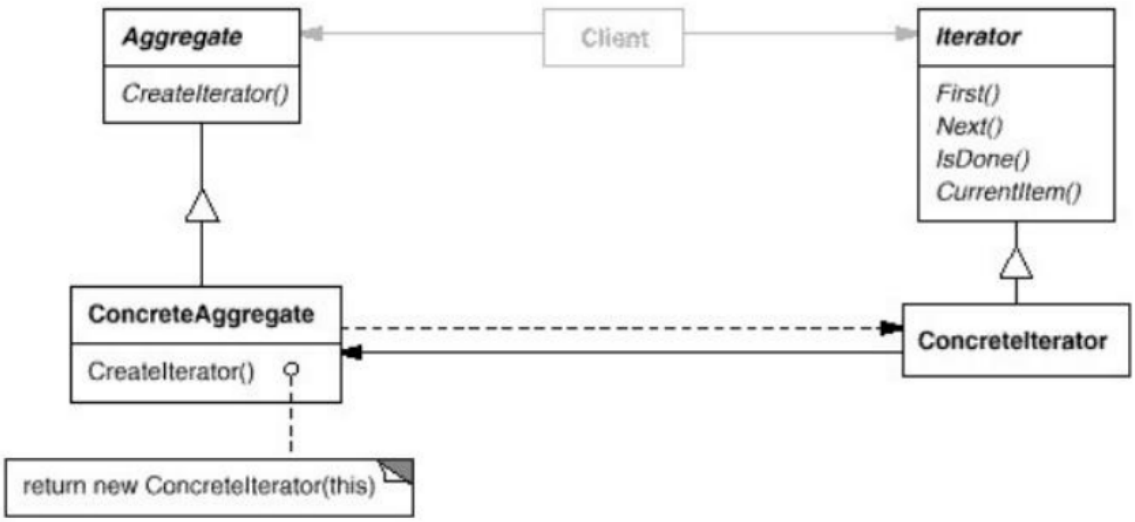
基本概念

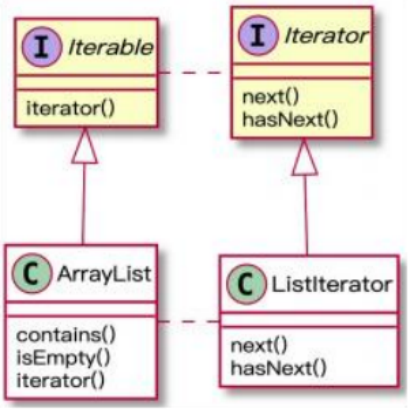
- **使用场景**：适合同一个业务步骤有多套可互换的实现（比如：不同排序/不同折扣/不同路径规划/不同压缩加密/不同风控规则），并且希望运行时可切换、避免 if-else/switch 越写越大、也避免修改老代码引入风险
- **使用原因**：把一件事的不同算法/不同做法抽成一个统一接口 Strategy，Context 只依赖接口，不关心具体实现；需要换算法时，换一个 Strategy 实例就行，不用改 Context 的代码
- **使用流程**：客户端先选一个具体策略（ConcreteStrategyA/B/C），把它交给 Context 保存；当调用 Context.Contextinterface() 时，Context 内部会转而调用当前策略的 Strategy.Algorithminterface() 来完成实际算法；需要换算法就把 Context 持有的策略替换掉，再调用同一个 Contextinterface()，行为就变了

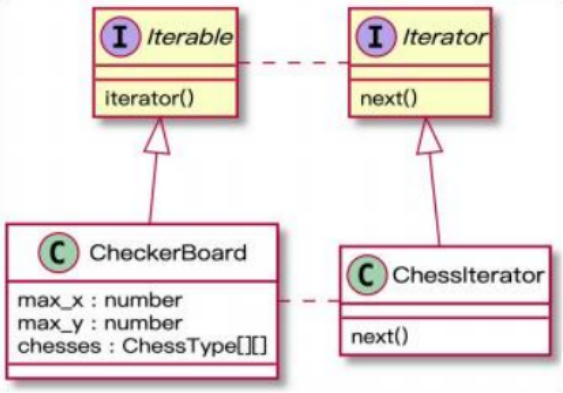
主要角色

- **Strategy（接口/抽象类）**：定义算法的统一入口
  - `Algorithminterface()`：算法/策略的执行入口，这里不写具体业务，只定义需要实现的能力
- **ConcreteStrategyA / B / C（具体策略）**：不同算法的具体实现
  - `Algorithminterface()`：各自实现各自的算法细节，真正的业务变化点
- **Context（上下文）**：持有一个 Strategy，对外提供稳定的业务入口，让调用方不用管具体算法细节
  - `Contextinterface()`：对外的统一调用入口（比如“计算价格”“执行任务”等），内部会委托给 `strategy.Algorithminterface()`
  - `Context -> Strategy`：表示 Context 依赖接口，而不是具体的 ConcreteStrategy 类

Iterator







基本概念

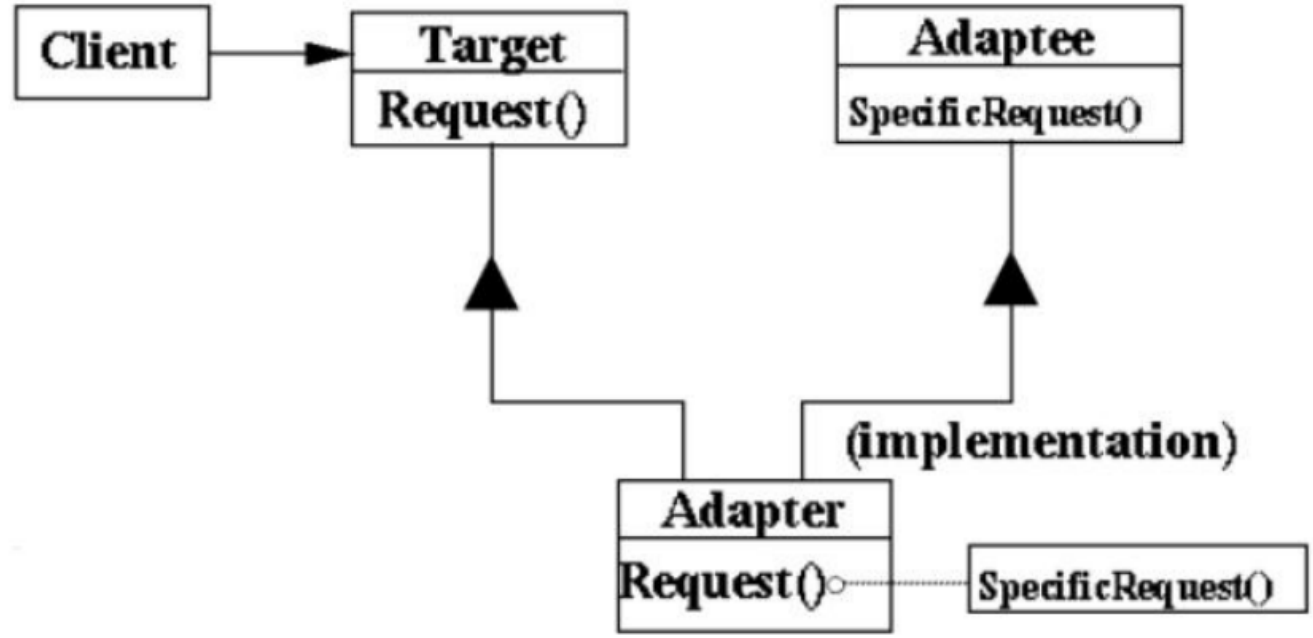
- **使用场景**：你有很多不同类型的集合（ArrayList、链表、二维棋盘、树等），都希望用同一套方式遍历；或者你想隐藏内部存储细节、避免客户端写一堆下标/指针逻辑；或者你想提供多种遍历方式（正序/倒序/跳步/过滤遍历）而不改容器代码。
- **使用原因**：迭代器模式把怎么遍历一个集合/容器的逻辑从容器本身里拆出去，封装到一个迭代器对象里，让客户端用统一的遍历接口访问元素，而不暴露容器内部数据结构（数组、链表、二维表、树.....）。
- **使用流程**：客户端拿到一个聚合对象 Aggregate，先向它要一个迭代器 `CreateIterator()`，然后在迭代器上循环：先定位到起点 `First()`，每次用 `IsDone()` 判断是否结束，用 `CurrentItem()` 取当前元素，并用 `Next()` 推进到下一个位置，直到遍历完成，客户端从头到尾都不需要知道容器内部是数组还是二维表。

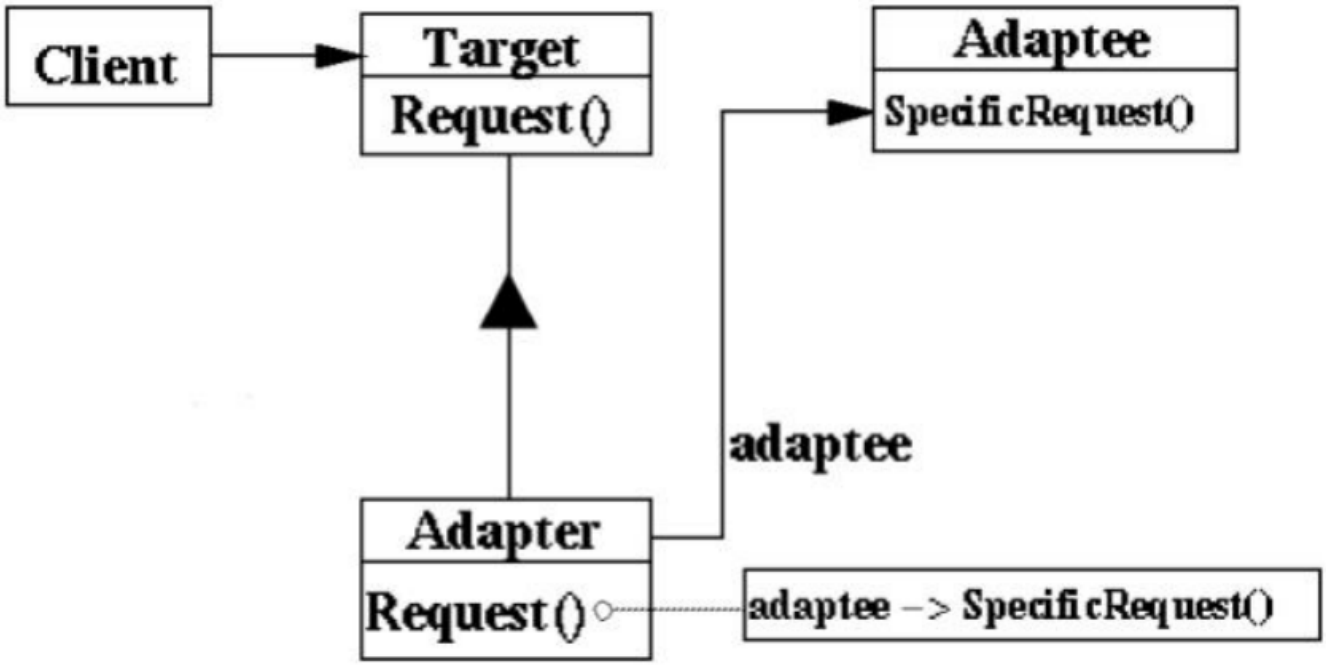
主要角色

- **Aggregate（聚合接口）**：用来提供创建迭代器的抽象接口，让客户端只依赖抽象
  - `CreateIterator()`：工厂方法，负责声明这个接口
- **ConcreteAggregate（具体聚合）**：实现创建迭代器的具体逻辑和实际存储数据（比如说 `ArrayList`）
  - `CreateIterator()`：具体实现这个方法：从集合对象里，创建并返回一个专门用来遍历它的迭代器对象
- **Iterator（迭代器接口）**：规定怎么遍历集合里的元素的抽象接口
  - `First()`：把游标移动到第一个元素（初始化起点）。
  - `Next()`：游标前进到下一个元素。
  - `IsDone()`：是否遍历结束（true 表示结束）。

    - `CurrentItem()`：返回当前游标指向的元素。
- **ConcreteIterator（具体迭代器）**：实现具体的以上四种（或者更多）遍历规则，保存遍历状态，从 Aggregate 里面读取数据

Adapter





基本概念

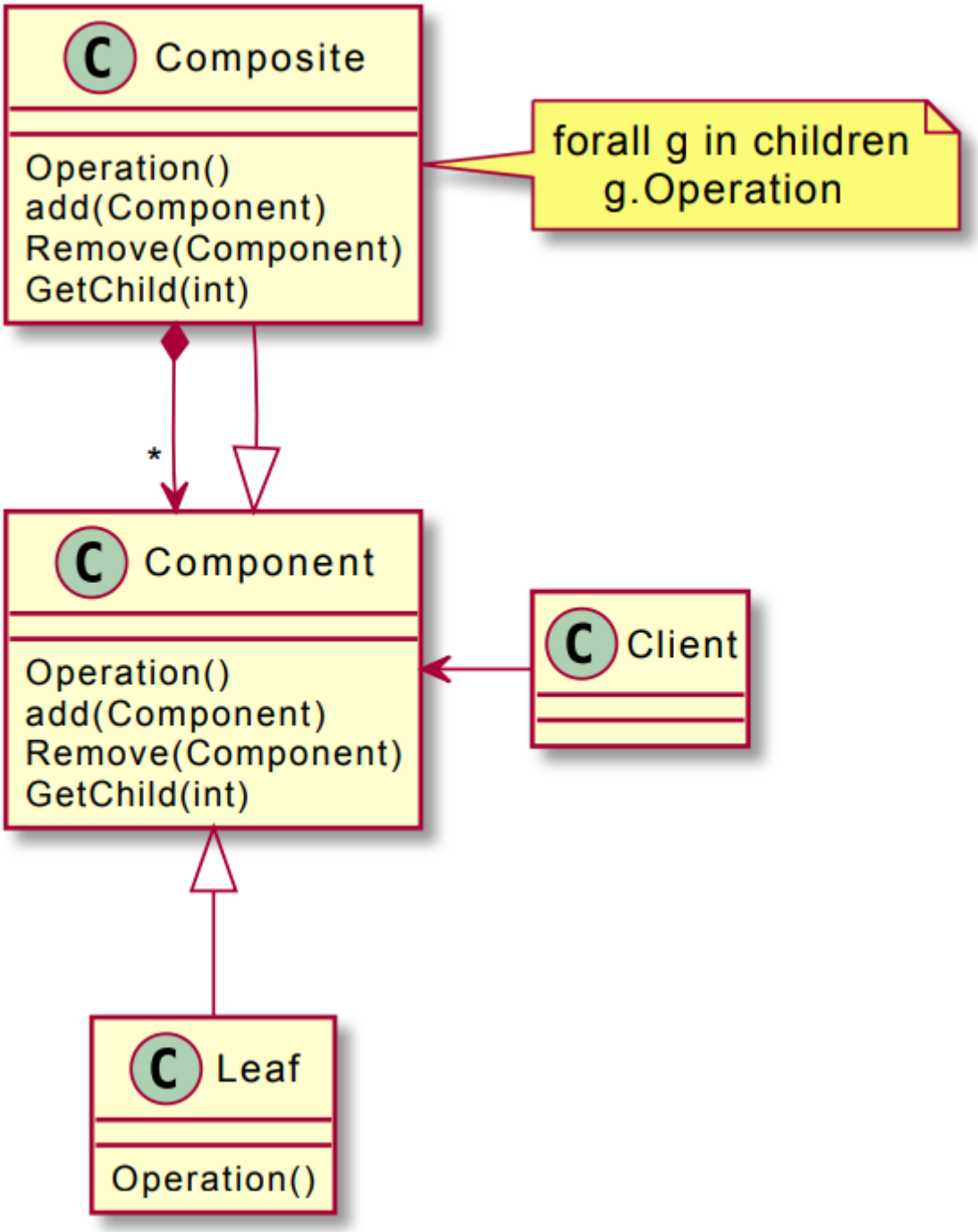
- **使用场景**：需要复用一段旧代码/第三方库 Adaptee，但它的接口和系统期望的接口 Target 不一致；又或者正在做系统重构/接口升级，老接口还在用，新实现接口不同，这时用 Adapter 让调用方不改、实现方也能继续用
- **使用流程**：Client 调用 `Target.Request()`，此时实际传进来的是 Adapter（它实现了 Target），Adapter 在 `Request()` 里把调用转成 `Adaptee.SpecificRequest()`，然后 Adaptee 完成真正的工作，结果由 Adapter 再转回给 Client

- **两种适配器的区别**：
  - **Class Adapter**：继承了 Adaptee，在 `Request()` 里面直接调用了 `SpecificRequest()`
  - **Object Adapter**：引用了一个 Adaptee 的对象，然后使用 `adaptee.SpecificRequest()`，实现调用

主要角色

- **Client**：调用方。它只依赖 Target，所以它只会调用 `Request()`，完全不知道 `SpecificRequest()` 这回事
- **Target**：定义 Client 想要的调用方式
  - `Request()`：给 Client 一个统一的调用入口
- **Adaptee**：真正干活的类，但接口长得不对，一个 Adaptee 可以被多个 Adapter 适配
  - `SpecificRequest()`：真正要干的事情的业务逻辑
- **Adapter**：把 Client 想要的接口翻译成 Adaptee 能听懂的 SpecificRequest 调用，一个 Adapter 一般只对应一个 Adaptee
  - `Request()`：转发调用的具体逻辑（直接调用父类的 `SpecificRequest()` 或者调用对象的 `adaptee.SpecificRequest()`）

Composite



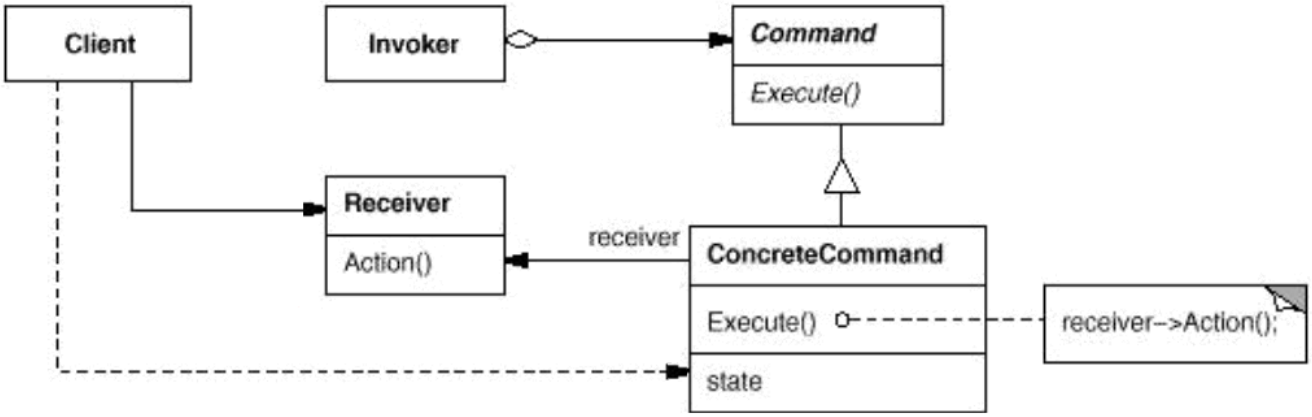
基本概念

- **使用场景**：当数据天然是树形结构，并且希望对“叶子”和“子树”执行同一类操作时就用它，比如：文件系统（文件/文件夹）、UI 组件树（按钮/面板）、组织架构（员工/部门）、菜单（菜单项/子菜单）、场景图等
- **使用原因**：它的核心目的是：把“单个对象（Leaf）”和“对象的组合（Composite，里面装一堆子节点）”用同一种接口（Component）对待，让客户端（Client）不需要写“如果是叶子就怎么做、如果是容器就怎么做”的分支逻辑，只需要对着 Component 调 `Operation()` 就行
- **使用流程**：Client 手里只拿到 Component 引用并调用 `Operation()`；如果这个引用实际是 Leaf，就直接执行叶子自己的业务；如果实际是 Composite，它会实现自己的 `Operation()` 的逻辑，并且把 `Operation()` 转发/递归到自己维护的 children 列表里，也就是图里那句 `for all g in children: g.Operation()`，从而让一次调用沿着整棵树走下去，对每个节点都完成相同语义的操作。

主要角色

- **Component（抽象构件）**：统一接口，Client 只依赖它
  - `Operation()`：对外暴露的统一操作。无论叶子还是组合都能调这个
  - `add(Component)` / `Remove(Component)` / `GetChild(int)`：树结构管理相关接口。图里把这些也放在 Component 里，意味着"透明组合"（客户端看起来所有节点都能 add/remove/getChild），但叶子一般不支持这些
- **Leaf（叶子）**：树的末端节点，不含子节点
  - `Operation()`：实现业务逻辑的地方（叶子自己的具体逻辑）
- **Composite（组合节点/容器）**：能装子节点的节点，内部维护 children
  - `add(Component)`：把子节点挂到自己下面
  - `Remove(Component)`：从自己下面删掉某个子节点
  - `GetChild(int)`：按下标拿某个子节点（有时也会提供按名字查）。
  - `Operation()`：和 `Leaf.Operation()` 一样，是实现 Composite 节点的业务逻辑的地方，但是还会遍历 children 并对每个孩子调用 Operation()
  - **Client（客户端）**：只面向 Component 编程。它不关心你给它的是 Leaf 还是 Composite，反正调 `Operation()` 就能得到正确效果

Command



基本概念

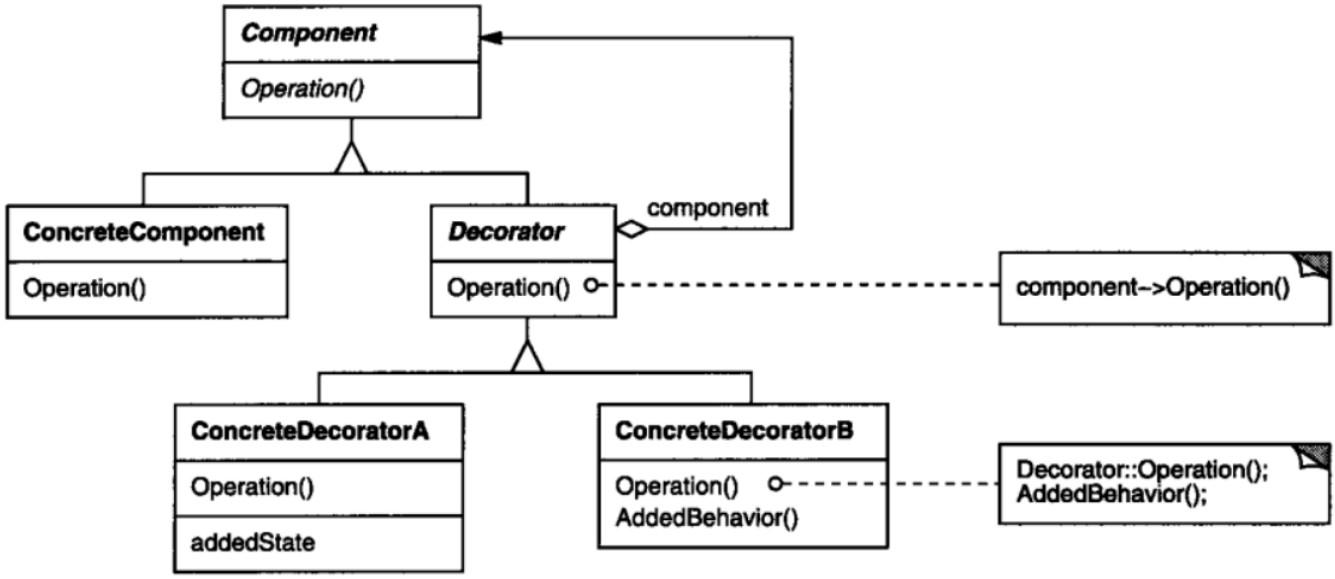
- **使用场景**：适合用在"发起请求的人（比如按钮/菜单/快捷键/队列/定时器）不想也不该知道具体怎么做，只负责把请求封装成一个对象交给别人执行"的场景：典型就是 GUI 按钮绑定操作、操作队列/日志、可撤销/重做（把命令存历史）、宏命令（多个命令组合）、异步任务/延迟执行、远程调用封装等
- **使用流程**：Client 先创建 Receiver（真正干活的对象）和 ConcreteCommand（把"要做什么"+"谁来做"封装起来），再把这个命令交给 Invoker；之后 Invoker 在合适的时机调用 `command.Execute()`，而 `ConcreteCommand.Execute()` 内部会转而调用 `receiver.Action()` 去完成真实业务（如果需要，命令里还会保存 state 作为执行参数或用于撤销/重做）

主要角色

- **Command（命令接口/抽象类）**：统一调用接口，声明 `command.Execute()`
  - `command.Execute()`：一个抽象的接口，Invoker 里面会写死只认这个接口
- **ConcreteCommand（具体命令）**：实现 Execute()，其中一般包含：
  - **receiver**：指向真正干活的对象

    - **state**：执行所需的参数/上下文（例如 “把灯开到 80%”、或者撤销需要的旧值）
    - `Execute()`：把调用转发给 receiver，也就是图右边那句 `receiver->Action()`
- **Receiver（接收者）**：完成工作的逻辑入口
    - `Action()`：具体的业务逻辑
- **Invoker（调用者/触发者）**：引用了个 Command 对象，在某个时机（按钮点击、定时触发、队列消费）调用 ``Execute()（也就是把命令转发给 Receiver，让 Receiver 执行 Action()`）。Invoker 不关心 Execute 里面做了什么，它只负责触发
- **Client**：Client（客户端/装配者）：负责把对象组装起来：创建 Receiver，创建 `ConcreteCommand(receiver, state)`，再把命令塞给 Invoker

Decorator



基本概念

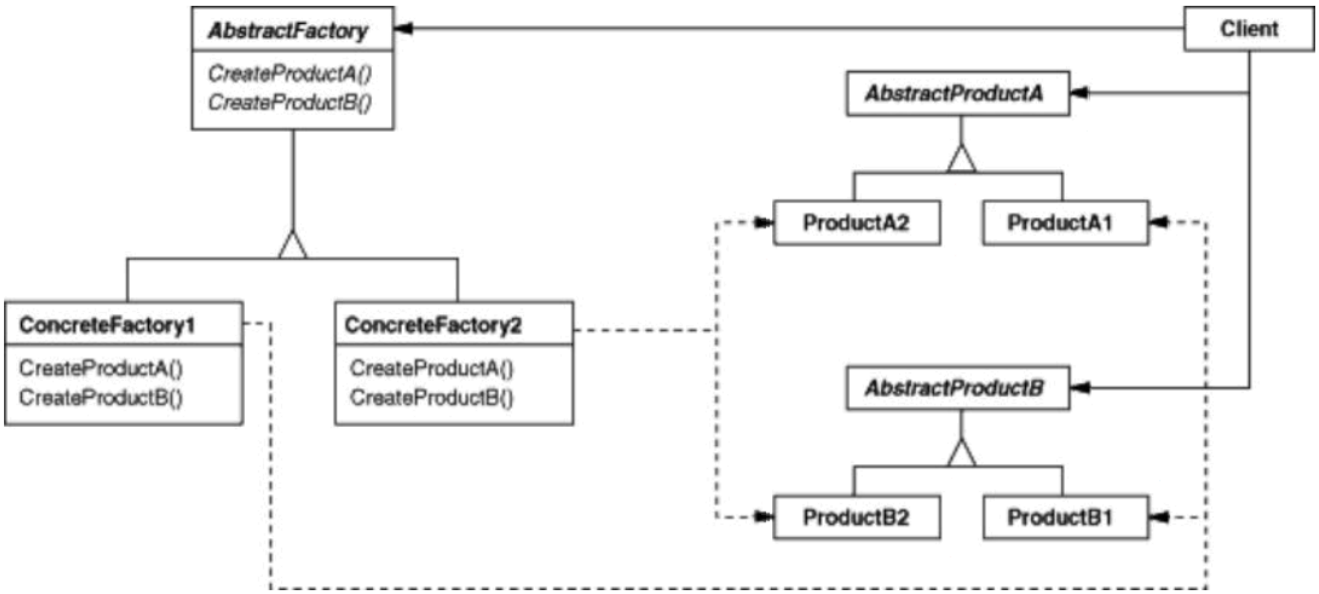
- **使用场景**：在不修改原有类、不通过继承爆炸的前提下，给对象动态地叠加新功能。适合“功能可选、可叠加、顺序有关”的场景，比如：IO 流、日志/鉴权/缓存横切功能、UI 组件外观增强等
- **使用流程**：客户端只面向 Component 编程；先创建一个 ConcreteComponent，再用一个或多个 Decorator 把它包起来；调用 `Operation()` 时，Decorator 的 `decorator.operation()` 会用内部引用的 Component 对象调用（转发） `component.Operation()`，然后继承 Decorator 的子类 ConcreteDecorator 也会进行自己添加的逻辑，在进行自己的逻辑之前使用 `super.Operation()` 完成父辈的逻辑

主要角色

- **Component**：抽象 Component 统一接口，定义了 `Operation()`。意义在于：无论是原始对象还是装饰后的对象，对外看起来都一样
    - `Operation()`：抽象的 `Operation()` 方法
- **ConcreteComponent**：干原始工作的类，实现 `Operation()` 的基础行为。它不知道装饰器的存在。
    - `Operation()`：原始的业务逻辑
- **Decorator**：抽象装饰器，也实现 Component，并持有一个 Component component 的引用
    - `Operation()` ：把调用转发给 `component.Operation()`，让原始的业务逻辑执行，装饰器添加的业务逻辑会在 Decorator 的子类里面执行
- **ConcreteDecoratorA / ConcreteDecoratorB**：具体装饰器，重写 `Operation()`

- `Operation()`：调用 `super.Operation()` 执行父辈逻辑、执行自己要增加的业务逻辑

Abstract Factory



基本概念

- **使用场景**：需要创建一整套“相互匹配/同一风格”的产品族（比如同一套 UI 皮肤的 Button+TextBox、同一套数据库驱动的 Connection+Command、同一品牌的零件A+零件B），并且希望切换“整套风格”时只换一个工厂对象，客户端代码完全不改；同时也避免客户端到处 `new ProductA1()` 这种写死具体类
- **使用流程**：Client 只依赖 AbstractFactory 和抽象产品接口 AbstractProductA/B；运行时 Client 拿到某个 ConcreteFactory1/2，然后分别调用 `CreateProductA() / CreateProductB()` 得到一套同族的 ProductA1 + ProductB1 或 ProductA2 + ProductB2（注意，这里没法拿到 A1 + A2，B1 + B2 这种，因为抽象工厂做的事就是从 A1/B1 换到 A2/B2；Client 后续只通过 AbstractProductA/B 的抽象方法使用产品，完全不知道也不关心具体类是什么。

主要角色

- **AbstractFactory（抽象工厂）**：定义了“这一整套产品族里有哪些产品要造”，但不决定造哪一种具体实现
  - `CreateProductA()`：创建 A 类抽象产品（返回 AbstractProductA）。
  - `CreateProductB()`：创建 B 类抽象产品（返回 AbstractProductB）。
- **ConcreteFactory1 / ConcreteFactory2（具体工厂）**：每个具体工厂负责生产同一族的配套产品（A1 配 B1；A2 配 B2）
  - `ConcreteFactory1.CreateProductA()`：返回 ProductA1
  - `ConcreteFactory1.CreateProductB()`：返回 ProductB1
  - `ConcreteFactory2.CreateProductA()`：返回 ProductA2
  - `ConcreteFactory2.CreateProductB()`：返回 ProductB2
- **AbstractProductA / AbstractProductB（抽象产品）**：Client 依赖的能力接口，对产品进行操作。一般会有：
  - `AbstractProductA.operationA()`：A 产品对外提供的功能
  - `AbstractProductB.operationB()`：B 产品对外提供的功能

- **ProductA1 / ProductA2、ProductB1 / ProductB2（具体产品）**：实现各自抽象产品接口，提供不同风格/不同实现细节
- **Client（客户端）**：只拿**工厂接口**创建产品，并只用**抽象接口**调用产品能力；切换产品族时，仅替换**传入的工厂对象**

具体使用场合

Visitor模式：编译器AST遍历、报表生成器、对象序列化、文档导出多格式，这种需要对固定结构添加多种操作的场合

Observer模式：股票行情推送、微信公众号订阅、GUI事件监听、MVC视图更新，这种一对多状态同步通知的场合

Strategy模式：支付方式选择（支付宝/微信/银行卡）、地图路径规划算法、图片压缩算法切换、促销折扣计算，这种同一业务有多种算法可切换的场合

Iterator模式：Java集合遍历、数据库ResultSet、文件目录树遍历、分页查询，这种需要统一遍历不同数据结构的场合

Adapter模式：旧系统接口改造、第三方SDK封装、数据格式转换器、充电器转接头，这种接口不兼容需要转换的场合

Composite模式：Windows文件管理器、公司组织架构、XML/DOM树、Swing/JavaFX组件树，这种树形结构需要统一处理的场合

Command模式：Word撤销/重做、线程池任务队列、智能家居遥控器、餐厅点餐系统，这种请求封装成对象延迟执行的场合

Decorator模式：Java IO流（BufferedReader包装FileReader）、SpringAOP日志/事务、HttpServletRequest增强、咖啡加糖加奶，这种动态叠加功能不改原类的场合

Abstract Factory模式：跨平台UI库（Windows/Mac风格）、数据库驱动（MySQL/Oracle）、游戏皮肤切换、云平台服务（AWS/阿里云），这种需要创建成套相关对象的场合

画 UML 类图

基本画图规范

类的表示

```
┌──────────────────┐
│   ClassName      │  ← 类名（抽象类用斜体，接口标<<interface>>）
├──────────────────┤
│ -attribute       │  ← 属性（-私有 +公有 #保护）
├──────────────────┤
│ +operation()     │  ← 方法
└──────────────────┘
```

关系表示

- **实现接口**：- - - -▷ （虚线+空心三角）
- **继承**：———▷ （实线+空心三角）
- **关联/依赖**：———→ （实线+普通箭头）
- **组合**：◆——— （实心菱形，表示强拥有，整体销毁部分也销毁）
- **聚合**：◇——— （空心菱形，表示弱拥有，整体销毁部分可独立存在）

画图要点

1. **使用场景化命名**：不要只写 `Element`/`ConcreteElement`，要根据题目写 `Shape`/`Circle`/`Rectangle`
2. **标注关键方法**：特别是模式的核心方法（如 `accept()`、`visit()`、`execute()`、`notify()`）
3. **体现模式结构**：按照对应模式的标准 UML 结构画，角色要齐全
4. **关系要清晰**：箭头方向要正确，关系类型要准确

示例：图形编辑器问题

题目：图形编辑器需要对不同图形（圆形、矩形、三角形）进行渲染、导出XML、计算面积等操作，且未来可能添加更多操作。

分析：

- 关键特征：对象类型固定（三种图形），操作经常变化（渲染、导出、计算，未来还可能更多）
- 判断：**Visitor 模式**

UML 方案：

```
          <<interface>>
           ┌──────────────┐
           │    Shape     │
           ├──────────────┤
           │+accept(v)    │
           └──────▲───────┘
                  │
      ┌───────────┼──────────┐
      │           │          │
┌─────┴────┐ ┌────┴─────┐ ┌──┴──────────┐
│ Circle   │ │Rectan.   │ │ Triangle    │
├──────────┤ ├──────────┤ ├─────────────┤
│+accept   │ │+accept   │ │+accept(v)   │
└──────────┘ └──────────┘ └─────────────┘

     <<interface>>
     ┌──────────────────┐
     │ ShapeVisitor     │
     ├──────────────────┤
     │+visit(Circle)    │
     │+visit(Rectangle) │
     │+visit(Triangle)  │
     └────────▲─────────┘
              │
```

```
        ┌─────────┴─────────┐
        │         │         │
        │         │         │
    ┌───┴───┐   ┌─┴─────────┐
    │Render   │ │XMLExport│
    │Visitor  │ │Visitor  │
    ├─────────┤ ├─────────┤
    │+visit() │ │+visit() │
    └─────────┘ └─────────┘
```

说明：

- 选择 Visitor 模式，因为图形类型固定但操作经常变化
- Shape 对应 Element，Circle/Rectangle/Triangle 对应 ConcreteElement
- ShapeVisitor 对应 Visitor 接口，RenderVisitor/XMLExportVisitor 对应 ConcreteVisitor
- 新增操作只需添加新 Visitor，不修改 Shape 类，符合开闭原则